

treasuryforge

Agente de tesorería cripto local-first, autónomo, con gobierno de riesgo cuantitativo

Detalle técnico del sistema

Versión: rama `harden/mutation-coverage` · 2026-06-18 · Python 3.12 (core stdlib-only) · 613 tests, cobertura 93.0%, gate de mutación ≥85% (money-path) · **riel HL probado con dinero real** · **motor carry-engine** + **cross-venue**

Este documento describe la arquitectura, las capas de gobernanza y seguridad, el ambiente de validación y el estado real del sistema — incluyendo sus **limitaciones honestas**. La premisa central no es prometer retorno; es **no hacer estupideces con dinero real**.

1 · Filosofía y principio rector

Una sola regla gobierna todo el diseño:

el AGENTE propone → la POLÍTICA dispone (reglas duras) → el WALLET ejecuta dentro de límites

- El "cerebro" (estrategia / LLM) **nunca se confía**: puede alucinar, emocionarse, sobre-interpretar datos o ser víctima de prompt-injection vía datos de mercado.
- Entre la intención propuesta y cualquier ejecución se interpone `policy.py`, una capa de reglas **no negociables** que el agente no puede modificar (config congelada).
- Core stdlib-only** (cero dependencias, cero red, cero llaves, cero fondos en el núcleo); las dependencias (keyring, SDK de Hyperliquid) viven en cuarentena en los adaptadores.
- El sistema está diseñado para **decir NO**: rechaza estrategias que no superan el tribunal de riesgo, y se cierra en falla (*fail-closed*) ante cualquier incertidumbre.

2 · Arquitectura por capas

Capa	Qué hace	Módulos
Percepción	señales mecánicas (probabilidad/edge)	<code>signals/funding</code> , <code>cointegration</code> , <code>pairs</code> , <code>linalg</code>
Decisión	propone una intención (comprar/vender/hold)	<code>agent.py</code>
Gobernanza	política dura + tribunal de riesgo cuantitativo	<code>policy.py</code> , <code>risk/*</code>
Ejecución	convierte intención aprobada en orden, idempotente	<code>executor</code> , <code>exchanges/*</code> , <code>orders.py</code>
Seguridad	preflight, watchdog, staleness, supervisor	<code>preflight</code> , <code>watchdog</code> , <code>staleness</code> , <code>live</code> , <code>service</code>
Durabilidad	WAL crash-safe + log de auditoría con hash-chain	<code>journal.py</code> , <code>audit.py</code>

El `Runner` determinista del simulador es el **núcleo validado** (no se toca). La ruta en vivo lo envuelve con el `LiveSupervisor` y el `Service` *fail-closed*.

3 · Motor de política — las 8 reglas duras

Evaluadas en orden; la primera que falla deniega la intención. Son la muralla entre la intención y la ejecución:

- Kill switch** — paro global, deniega todo.
- Circuit breaker** — si el equity cae más allá de `max_drawdown`, traba y se queda trabado.
- Staleness gate** — deniega si los datos son más viejos que el presupuesto (*fail-closed*).
- Allowlist** — solo activos aprobados.
- Cap de notional por tx** (+ piso mínimo del exchange).
- Rate limit** — máximo de trades por ventana.
- Spend budget** — máximo de valor acumulado por ventana (cierra el drip-drain que un límite por conteo deja).
- Solvencia** — nunca propone lo que el wallet no puede pagar.

La config es **congelada** (frozen dataclass) y el estado latchado (breaker, ventanas) es **snapshot/restore-able** vía `journal.py` : un crash no puede des-trabar silenciosamente el breaker ni re-anclar el piso de drawdown a un equity post-pérdida.

4 · Framework cuantitativo de riesgo

El núcleo de "¿debo desplegar esta estrategia, y con cuánto?". Cadena de razonamiento:

```
MEDIR (E, σ, distribución) → VALIDAR (DSR) → DIMENSIONAR (f = DSR · Kelly · ½) → SOBREVIVIR (ruina)
```

- **Expectancy:** $E = p(g-c) - (1-p)(l+c)$. La frecuencia amplifica el *signo* de E (edge negativo × alta frecuencia = ruina rápida).
- **Crecimiento geométrico:** $g \approx E - \sigma^2/2$ (drag de varianza). Se dimensiona por crecimiento log, no por el promedio aritmético.
- **Kelly fraccional:** $f^* = E/\sigma^2$, usado a fracción $\lambda \approx \text{DSR}$ (Bayesian-Kelly bajo incertidumbre de modelo: el mismo DSR que detecta overfitting es el multiplicador de tamaño).
- **Deflated Sharpe Ratio (DSR):** corrige el Sharpe por nº de configs probadas (selection bias), tamaño de muestra, skew y kurtosis. Responde: "dado que probé N estrategias, ¿esto es mejor que la mejor suerte?".
- **Purged/embargoed CV:** validación cruzada que evita fuga temporal entre folds.
- **Monte-Carlo risk-of-ruin:** block-bootstrap (preserva rachas), con shocks de cola y régimen adverso. Demostró que full-Kelly es ruina casi segura.

Las compuertas (gates) de despliegue

#	Gate	Criterio
0	edge_is_real	$\text{DSR} \geq \text{min_dsr}$ (default 0.60)
1	edge	$E[r] > \text{costo} + \text{margen}$
2	geometric_growth	$E[\ln(1+f \cdot r)] > 0$
3	kelly_cap	$\text{leverage} \leq \lambda \cdot \text{Kelly} \text{ y } \leq \text{cap duro}$
4	risk_of_ruin	$P(\text{ruina estresada}) < \epsilon$
5	expected_drawdown	$E[\text{maxDD}] < \text{umbral}$

5 · Capas de gobernanza (ciclo de vida de la estrategia)

- **Reporte de decisión** — veredictos nombrados: `NO_EDGE` / `EDGE_IS_NOT_RELIABLE` / `PATH_TOO_DANGEROUS` / `DEPLOY_SMALL`, con métricas crudas, validación, sizing y riesgo.
- **Detector de gap backtest-vs-live** — compara expectativa de backtest vs paper-live, slippage realizado vs modelado, fill-rate; veredicto desde `EDGE_DECAYED` hasta `APPROVE_STAGE_1`.
- **Escalera de etapas** — DEAD → PAPER_LIVE → MICRO (1%) → SMALL (3%) → NORMAL (8%) → CAPPED (20%). Prioridad KILL > DEMOTE > PROMOTE > HOLD. Una estrategia gana capital un escalón a la vez y se degrada/mata si su comportamiento en vivo diverge.
- **Drift kill-switch (Page-Hinkley)** — detector secuencial de cambio: caída de expectativa, subida de slippage o DD fuera de banda → KILL permanente.
- **Cementerio de estrategias** — JSONL append-only: cada rechazo guarda hipótesis, modo de fallo, condición de reactivación y un *fingerprint* (atrapa el mismo cadáver disfrazado).
- **Hashes de reproducibilidad** — hash canónico de datos/config/estrategia/reporte para asegurar verdad histórica.

6 · Capa de seguridad de ejecución (los "no-regrets")

Construida y endurecida ANTES de cualquier flujo de orden real. Todo inyectable (sin reloj/red/llaves en tests).

6.1 — IdempotentOrderManager (exactly-once)

El fallo más peligroso en vivo no es una mala estrategia; es **duplicar una orden** por timeout, crash o retry. Se enforza con dos mecanismos:

- **origin_id estable** por intención lógica, persistido en el journal → todo retry/reinicio reusa el mismo id.
- **reconcile-before-retry**: ante cualquier resultado incierto (timeout/5xx) o tras un crash, se reconcilia por `origin_id` ANTES de actuar; un fill se registra una vez, y solo un no-fill confirmado se reintenta (con el mismo id, que el exchange deduplica entre órdenes activas).

```
PLACING → SUBMITTED → FILLED      (camino feliz)
PLACING → REJECTED                  (rechazo determinista, nunca aterrizó)
PLACING → UNKNOWN → FILLED | OPEN   (timeout/5xx, resuelto por reconcile)
FILLED y REJECTED son terminales; submit() sobre un id terminal replica el resultado.
```

6.2 — DeadMansSwitch (watchdog cliente-side)

Bitso no tiene `cancelAllAfter` server-side, así que se construye el equivalente: el loop debe `beat()` cada `timeout_s`; un supervisor dispara el fail-safe (cancelar órdenes resting + trabar kill-switch) **exactamente una vez** y queda trabado (fail-closed, reset manual). El heartbeat se persiste atómicamente: un supervisor fresco trips sobre un latido viejo (el bot murió con órdenes vivas). El fail-safe corre ANTES de persistir el trip, así un crash a media cancelación re-dispara en vez de saltársela.

6.3 — Preflight fail-closed (arranque seguro)

11 precondiciones; si **cualquiera** falla → estado `SAFE_HALTED`, nunca "best effort": modo explícito, config cargada, journal escribible, credenciales presentes, allowlist activa, caps cargados, kill-switch limpio, reloj sano, skew de reloj acotado, feed de datos fresco, exchange alcanzable.

6.4 — StalenessGate (runtime: reloj + frescura)

Rehúsa operar si: el wall-clock saltó hacia atrás, el drift wall/monotonic excede presupuesto (NTP step / VM pause), o el dato más fresco supera su budget. Provee el `data_age_ns` real que la regla de staleness de la política consume; `WINDOW_24H_NS` para ventanas reales de 24h.

6.5 — LiveSupervisor + Service + systemd

El **LiveSupervisor** une todo en un `step()` gobernado: late el watchdog → rehúsa con dato viejo/reloj raro → la política dispone → ruta la orden por el manager idempotente. El **Service** es el proceso externo fail-closed: preflight-gate, reconcilia órdenes in-flight ANTES del loop (recuperación de crash, nunca re-coloca), exit limpio si trips el dead-man, y cierra ante CUALQUIER excepción (dispara cancel+kill). Códigos de salida que la unidad de systemd usa:

Código	Significado	¿Reinicia?
10 <code>SAFE_HALTED</code>	preflight rechazó	NO — humano arregla el entorno
20 <code>DEAD_MAN</code>	el dead-man disparó	NO — humano inspecciona
30 <code>ERROR</code>	falla inesperada	SÍ — el arranque re-corre preflight + reconcilia primero

6.6 — Riel de ejecución Hyperliquid (3 gates, dinero real probado)

Construido en el orden seguro, cada gate antes del siguiente:

1. **Dry-run gate** (keyless) — arma el payload L1 exacto con formato fiel al SDK (size→szDecimals, precio→5 sig-figs, strings normalizados) y lo valida contra el piso de \$10 del venue y el cap de seguridad, **sin firmar ni mandar**.
2. **Sign-without-send** (VPS) — firma con la agent key, recupera el firmante y confirma que es uno de los agents **autorizados on-chain** (`extraAgents`, no solo el del frontend), **sin POST**.
3. **Orden viva** — vía el `IdempotentOrderManager`: `cloid` determinista (idempotencia nativa de HL → un retry reusa el mismo id y el venue deduplica), reconcile del fill *por cloid*, y close reduce-only (el piso de \$10 es regla de apertura, no de cierre).

La agent key (trade-only / no-withdraw) la provisiona el dueño en el VPS (`read -s` oculo → `env-file 600`), **nunca por chat**. Gotcha resuelto: la cuenta unificada de HL reporta el sub-ledger de perps en \$0 mientras el USDC vive en spot — el colateral se lee como perp + spot USDC.

7 · Estrategias, tribunal y shadow

7.1 — Funding-carry (delta-neutral)

Long spot + short perp (size-matched): el PnL de precio se cancela; siendo short del perp recibes funding cada intervalo cuando el rate es positivo. Datos reales de Hyperliquid: funding positivo regime-dependiente en ETH/BTC (~10.95% APR en picos), negativo en SOL/AVAX. **Veredicto del tribunal: RECHAZADA en backtest** (DSR ~0.24 — edge incierto). Es el único edge con base estructural real, así que pasa al shadow para el filtro en vivo.

7.2 — Stat-arb pairs (cointegración) + regime-gating

Engle-Granger con valores críticos Phillips-Ouliaris correctos, z-score rodante, dollar-neutral. Real-data: solo ~8/45 pares cointegran; los Sharpe altos eran artefacto de selección (DSR bajo, fold de CV negativo, ruina estresada alta). **RECHAZADO**. Se añadió el **regime-gating** que el research marcó como no-negociable: `variance-ratio` (reversión vs tendencia) + ADF rolling (detector de ruptura estructural) + banda de half-life + stop-z direccional. Medido honesto en datos reales: el gate corta 86% de los trades por régimen no-reversivo, **pero todos los DSR gated siguen ≤ 0.21** — el gate es una capa de RIESGO correcta (no tradear un spread roto), *no* un generador del alpha que no existe.

7.3 — MR_SCALP (mean-reversion corto) — construida para matarla

Idea de scalping z-score (hold ≤ 10 min) convertida en regla mecánica con 5 castigos obligatorios en el backtest (fill maker conservador, stop-first peor-caso, sin salida en la misma vela, métricas de adverse selection, gate edge/costo). Real-data HL: ETH/BTC **COSTS_EAT_EDGE** (edge bruto ya negativo); SOL parecía máquina de dinero (100% win, DSR 1.000) sobre 2 trades → **INSUFFICIENT_FREQUENCY**, el espejismo de muestra chica que el gate de frecuencia mató. Enterrada en el cementerio.

Que ninguna pase NO es un fallo — es el sistema funcionando. El tribunal mató a los perdedores Y resistió a los "ganadores" seductores de muestra chica. Ese es exactamente su trabajo.

7.4 — Basis spot-vs-perp (cash-and-carry)

Segunda estrategia, genuinamente distinta del funding-carry: en un perp el basis es el funding, así que ésta modela el stream EXTRA que el funding-carry ignora — la **convergencia del premium** (mark vs oracle). ENTER cuando el perp está rico (contango, $\text{premium} \geq \text{umbral}$), HOLD cobrando funding + ($\text{premium previo} - \text{actual}$) mientras revierte, EXIT al converger. Track record independiente.

7.5 — Shadow paper-book × 10 (el filtro honesto, en vivo)

Corre las señales sobre datos **en vivo** (Hyperliquid `/info`, keyless), simula la posición y acumula PnL hipotético — **sin dinero, sin llaves, sin venue**. Ahora **10 track records en paralelo**: funding-carry × 5 monedas (BTC/ETH/SOL/AVAX/ARB) + basis × 5 monedas. Cada uno persiste en su journal (acumula día a día, sobrevive reinicios). `shadow_status.py` agrega TODOS los ledgers en un board (intervalos, días, % en posición, funding/convergencia, net, Sharpe/DSR en vivo) convergiendo hacia el gate 0.60. Corre como `systemd timer` horario en el VPS. Multiplicar monedas/ estrategias multiplica la evidencia-por-hora a costo cero — la forma honesta de "validar más rápido" (no abrir posiciones a ciegas, que es apostar con EV negativo y no valida el edge).

8 · Ambiente de validación

8.1 — Los 5 procesos de escaneo (en Docker, todos verdes)

Proceso	Qué valida	Estado
ruff	lint + complejidad (C901, max 14)	PASS
mypy --strict	tipos estáticos (strict_equality, etc.)	PASS
bandit	seguridad (SAST)	PASS
pip-audit	CVEs en dependencias	PASS
pytest-cov	tests + cobertura ≥90%	PASS (92.9%)

8.2 — Testing de mutación (mutmut) — gate ≥85% por componente

La lección clave: **cobertura alta ≠ tests fuertes**. La mutación inyecta bugs y verifica que los tests los detectan (kill-score = matados/total). Sizing tenía 91% de cobertura pero 56% de mutación. Resultados:

Componente (core)	Mutación	Componente (seguridad/wiring)	Mutación
policy.py	94.8%	orders.py	90%
risk/ruin.py	96.6%	watchdog.py	92%
sizing.py	96%	preflight.py	91%
metrics.py	97%	staleness.py	89%
risk/stages.py	88%	live.py	97%
risk/drift.py	98.5%	service.py	100%
risk/graveyard.py	90%	run_live.py	≥85%
risk/live_gap.py	94%	shadow.py	97%

Los survivors residuales son *probablemente equivalentes* (anotaciones que no se evalúan, defaults None falsy≡False, ruido de epsilon ±1e-9 sub-nanodólar, o mutantes no-viables que rompen el import y mutmut malclasifica — excluidos con `# pragma: no mutate` justificado). Ningún mutante de comportamiento sobrevive.

8.3 — Tres niveles de fidelidad pre-prod

1. **Emuladores** (MockBitsoAPI, FakeExchange) — contrato HTTP, milisegundos.
2. **Mercado sintético** — jump-diffusion con regímenes (calm/storm/crash), stress de volatilidad.
3. **BD de datos reales** (`marketlab`) — candles/funding/L2 reales de Hyperliquid + modelo de fills walking-the-book.

Pipeline CI: GitHub Actions (matriz 3.11/3.12 + job de mutación), Docker, docker-compose + SonarQube.

9 · Estado en vivo

- **Bitso (spot)**: primera operación real ejecutada + reconciliada (20 MXN, BTC). Cadena completa probada: firma HMAC → IP-allowlist → place → reconcile → **fee-en-base verificada contra dinero real**. Key trade-only / no-withdraw / IP-locked al VPS.
- **Hyperliquid (perps)**: **RIEL DE EJECUCIÓN PROBADO END-TO-END**. Primera orden real: BUY \$11 ETH (0.0063 @ \$1755.40) → reconcile por cloid → close reduce-only → flat. **Round-trip completo por \$0.02** (1¢ fees + 1¢ drift de precio); colateral \$19.85 intacto. El rail test atrapó un bug real (size sin redondear) **ANTES** de mover dinero — el SDK lo rechazó pre-POST y la idempotencia + reconcile sostuvieron (cero posición, cero duplicado). Confirmó que la cuenta unificada respalda perps desde el USDC de spot. La agent key vive solo en el VPS.
- **Shadow**: 10 track records (5 funding-carry + 5 basis, monedas BTC/ETH/SOL/AVAX/ARB) en vivo, keyless, horario, automático en el VPS. Status on-demand vía `shadow_status.py`.

Dos ejes separados, no confundir. El **RIEL** de ejecución está PROBADO y de-riesgado (real, \$0.02). El **EDGE** de estrategia NO está confirmado — eso lo decide el shadow con *tiempo de mercado*, no con más trades. Abrir posiciones sin señal valida el riel (ya hecho), nunca la estrategia (sería apostar con EV negativo).

10 · Limitaciones honestas y veredicto

Ninguna estrategia ha pasado los gates todavía. Funding-carry y pairs fueron ambas RECHAZADAS (DSR bajo, ruina estresada alta). El bot, hoy, **no operaría nada** en vivo. No hay "máquina de dinero encendida".

- **Los edges retail son delgados:** ~3-4% APR (funding-carry), Sharpe OOS ~0.69 (pairs). **NO 15-20% mensual** — ese número es señal de Ponzi, no de trading.
- **El microarbitraje "centavos por segundo" fue desmentido** (deep-research de 106 agentes): spread real ~0.025%, ventana 146 ms solo-HFT, y el pitch textual = la firma del Ponzi Trade Coin Club (\$295M).
- **El valor del sistema NO es retorno**, es: honestidad (rechaza lo malo), control (auto-custodia, código auditable), y aprendizaje (entiendes y dueñas la máquina).

treasuryforge ya no intenta demostrar que puede ganar dinero. Demuestra que **puede no hacer estupideces**. Primero sobrevives; luego mides; luego rechazas casi todo; luego, si un edge sobrevive en vivo, le das micro-capital; después, quizá, escalas.

11 · Inventario de módulos

Módulo	Responsabilidad
types, agent, policy, executor, wallet, runner, market	núcleo del simulador (validado)
journal, audit	WAL crash-safe + log con hash-chain HMAC
sizing, backtest/{metrics, cost, cv, funding_backtest, pairs_backtest, mr_scalp_backtest}	dimensionamiento + backtesting con gates
risk/{ruin, stages, drift, graveyard, live_gap, provenance, report}	tribunal de riesgo + gobernanza
signals/{funding, cointegration, pairs, regime, basis, mr_scalp, linalg}	señales mecánicas + regime-gating
exchanges/bitso/{signer, errors, executor, client, transport, mock, validation}	adaptador Bitso (live)
exchanges/hyperliquid/{info, executor, wire, signer, live_executor, secrets}	adaptador Hyperliquid (riel de ejecución probado)
orders, watchdog, preflight, staleness	capa de seguridad de ejecución
live, service, run_live	supervisor + proceso fail-closed + endpoint
shadow, shadow_basis	paper-trading en vivo (funding + basis, sin fondos)
marketlab/{store, synthetic, fills}	BD de mercado + sintético + fills
deploy/*.service, *.timer	unidades systemd (servicio 24/7 + shadow horario)

12 · Motor "carry-engine": investigación y cross-venue (2026-06)

Reframe de "trading bot" a **motor de carry + plataforma de validación**. El cuello de botella no es el riel (probado, \$0.02 round-trip), es el **edge neto validado en vivo**. Cada feature = hipótesis con gate de matar/quedarse (ver `ROADMAP.md`).

12.1 — La cadena de investigación (cada capa redujo incertidumbre con datos)

Capa	Herramienta	Veredicto honesto
A · Medición	<code>pnl_decomposition</code> , <code>fill_model</code> , <code>carry_screener</code>	el net negativo = fee-bleed por churn (fees 10-64× el funding)
B1 · Universo	<code>universe.py</code> (230 perps)	HL capea funding en +10.95%; el costo NO ata a hold largo; la persistencia sí
C · Persistencia	<code>funding_persistence.py</code>	el funding CHOPEA: mediana de episodio 6-25h, solo 6-50% llega a break-even (~40h)
C2 · Predicción	<code>funding_predictor.py</code>	el chop es seleccionable: age-rule 17→39% y coin-selection 11→27% OOS
Backtest econ.	<code>carry_backtest.py</code>	fresh = -185 bps (ruina); age-rule = +4 bps (break-even, DSR 0)

Carry single-venue: SIN edge desplegable. El mejor selector solo llega a break-even; su valor es *evitar la ruina*, no ganar. Investigación cerrada con datos.

12.2 — Cross-venue: el primer candidato que sobrevive la rigor

`cross_venue.py` midió spreads de funding HL-vs-otro venue (delta-neutral: short el de funding alto, long el bajo). De 57 PAPER snapshot, la rigor (liquidez en ambos + persistencia + costo real) dejó **2 coins líquidos: XRP y ZEC**. Verificación cruda: el perp de XRP en HL tiene funding **persistente ~-30% APR** mientras Binance/OKX están en ~0 — spread real ~30% gross, venue-agnóstico (**HL es el outlier estructural**). Por qué persiste = **la fricción es el foso** (cuentas+colateral en 2 venues ≈ 2× capital, liquidación cruzada, KYC). Hallazgo: **Binance geo-bloqueado desde el VPS (451) → OKX** (alcanzable, mismo spread). Real pero operacionalmente duro; NO desplegable aún (falta modelar economía dual + forward-shadow).

12.3 — Shadows activos (paper, \$0, horario en VPS) y pronóstico

#	Shadow	Estado	Pronóstico / ETA de validación
1	Cross-venue XRP (HL-OKX)	EL MÁS PROMETEDOR — spread ~30%, sobrevivió toda la rigor; forward-test recién iniciado (age-rule esperando 24h)	2-4 semanas para confirmar persistencia + carry neto positivo en vivo
2	Cross-venue ZEC	spread ~33% pero la pata OKX/Binance es volátil (menos limpio)	2-4 semanas; vigilar estabilidad de la 2ª pata
3	Funding-carry × 11 coins	todos net negativo (-0.05 a -0.24%), cost/grs 1200-6400%; age-rule ahora gateando entradas	~1-2 días para DSR; ya sabido ≈break-even (confirmación)
4	Basis spot-perp × 11	casi todos 0-held (backwardation); sin edge en el régimen actual	espera contango; sin ETA de edge

Mejoras detectadas en los shadows: (a) validar forward que el age-rule evita el churn en vivo; (b) modelar economía real cross-venue (colateral dual + liquidación) para el APR neto sobre capital; (c) ampliar el escaneo cross-venue a más venues alcanzables.

12.4 — Calidad (5 procesos + mutación)

Escaneo unificado: **613 tests verdes**, ruff/mypy strict limpios, bandit 0 issues de severidad media+, cobertura **93.0%**, pip-audit sin CVEs. Mutación: el **money-path core sigue ≥85%** (intacto). Los ~12 componentes nuevos de analítica son tooling de medición; `cross_venue` ya endurecido a **89%** (gate pasado), los demás subidos a 72-81% en el primer pase (grind multi-pase en curso, infraestructura de mutación arreglada y lista).

12.5 — Status en vivo del forward-test (snapshot 2026-06-18 13:00 CEST)

Primeras ~13h de acumulación. **El forward-test ya entregó valor: atrapó variabilidad real antes de arriesgar capital.**

Shadow	Observación en vivo	Lectura honesta
Cross-venue XRP	spread colapsó de ~30% a ~1.1% APR en ~12h; 0 entradas (age-rule esperando 24h)	el age-rule evitó la pérdida ; el spread es más regime-dependiente de lo que el snapshot sugería — la pregunta real es <i>cuánto del mes está alto</i>
Cross-venue ZEC	spread ~16% APR, age 3/24, sin entrar	acumulando; vigilar

Funding-carry ETH	31 obs → Sharpe -32.8, DSR 0.000 (primer DSR real)	SIN edge confirmado con números, como predijo el backtest económico
Funding-carry ×11	todos net-negativo; los nuevos (ZEC/HYPE/ASTER) mejoraron cost/grs 1200%→338-686% al acumular funding pero siguen en pérdida	≈break-even at best; el age-rule ahora gatea entradas nuevas
Basis ×11	casi todos 0-held (backwardation); algunos whipsaws -0.08 a -0.31%	sin edge en el régimen actual

El candidato #1 (XRP) mostró que su spread no es constante. Sigue siendo el mejor que tenemos, pero la barra subió: hay que medir cuánto tiempo el spread está realmente alto. ETA de validación sin cambios: **2-4 semanas** de acumulación para ver la distribución del spread. Nada que tocar — el sistema mide la verdad solo, sin riesgo.